



Creación de un chatbot con IA capaz de buscar información en fuentes externas

Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:

Mario Martínez Turpín

Tutor/es:

Jose Antonio Miñarro Gimenez (tutor1)

Nombre Apellido1 Apellido2 (tutor2)

18 de Mayo de 2026



**Facultad
Informática
Universidad
Murcia**

Creación de un chatbot con IA capaz de buscar información en fuentes externas

Subtítulo del proyecto

Autor

Mario Martínez Turpín

Tutor/es

Jose Antonio Miñarro Gimenez (tutor1)

Departamento Informática y Sistemas

Nombre Apellido1 Apellido2 (tutor2)



Grado en Ingeniería Informática



UNIVERSIDAD DE
MURCIA



Murcia, 18 de Mayo de 2026

Preámbulo

Las principales razones por las que he realizado este trabajo son mi pasión por la inteligencia artificial, el afán de empezar a desarrollar por mi cuenta

Agradecimientos¹

Este trabajo no habría sido posible sin el apoyo y el estímulo de mi colega y amigo, Doctor Rudolf Fliesning, bajo cuya supervisión escogí este tema y comencé la tesis. Sr. Quentin Travers, mi consejero en las etapas finales del trabajo, también ha sido generosamente servicial, y me ha ayudado de numerosos modos, incluyendo el resumen del contenido de los documentos que no estaban disponibles para mi examen, y en particular por permitirme leer, en cuanto estuvieron disponibles, las copias de los recientes extractos de los diarios de campaña del Vigilante Rupert Giles y la actual Cazadora la señorita Buffy Summers, que se encontraron con William the Bloody en 1998, y por facilitarme el pleno acceso a los diarios de anteriores Vigilantes relevantes a la carrera de William the Bloody.

También me gustaría agradecerle al Consejo la concesión de Wyndham-Pryce como Compañero, el cual me ha apoyado durante mis dos años de investigación, y la concesión de dos subvenciones de viajes, una para estudiar documentos en los Archivos de Vigilantes sellados en Munich, y otra para la investigación en campaña en Praga. Me gustaría agradecer a Sr. Travers, otra vez, por facilitarme la acreditación de seguridad para el trabajo en los Archivos de Munich, y al Doctor Fliesning por su apoyo colegial y ayuda en ambos viajes de investigación.

No puedo terminar sin agradecer a mi familia, en cuyo estímulo constante y amor he confiado a lo largo de mis años en la Academia. Estoy agradecida también a los ejemplos de mis difuntos hermano, Desmond Chalmers, Vigilante en Entrenamiento, y padre, Albert Chalmers, Vigilante. Su coraje resuelto y convicción siempre me inspirarán, y espero seguir, a mi propio y pequeño modo, la noble misión por la que dieron sus vidas.

Es a ellos a quien dedico este trabajo.

¹Por si alguien tiene curiosidad, este “simpático” agradecimiento está tomado de la “Tesis de Lydia Chalmers” basada en el universo del programa de televisión Buffy, la Cazadora de Vampiros.<http://www.buffy-cazavampiros.com/Spiketesis/tesis.inicio.htm>

*A mi esposa Marganit, y a mis hijos Ella Rose y Daniel Adams,
sin los cuales habría podido acabar este libro dos años antes*²

²Dedicatoria de Joseph J. Roman en "An Introduction to Algebraic Topology"

*Si consigo ver más lejos
es porque he conseguido auparme
a hombros de gigantes*

Isaac Newton.

Declaración firmada sobre originalidad del trabajo

D./Dña. **Mario Martínez Turpín**, con DNI **49183039T**, estudiante de la titulación de **Grado en Ingeniería Informática** de la Universidad de Murcia y autor del TF titulado “**Creación de un chatbot con IA capaz de buscar información en fuentes externas**”.

De acuerdo con el Reglamento por el que se regulan los Trabajos Fin de Grado y de Fin de Máster en la Universidad de Murcia (aprobado C. de Gob. 30-04-2015, modificado 22-04-2016 y 28-09-2018), así como la normativa interna para la oferta, asignación, elaboración y defensa de los Trabajos Fin de Grado y Fin de Máster de las titulaciones impartidas en la Facultad de Informática de la Universidad de Murcia (aprobada en Junta de Facultad 27-11-2015)

DECLARO:

Que el Trabajo Fin de Grado presentado para su evaluación es original y de elaboración personal. Todas las fuentes utilizadas han sido debidamente citadas. Así mismo, declara que no incumple ningún contrato de confidencialidad, ni viola ningún derecho de propiedad intelectual e industrial

Murcia, a 18 de Mayo de 2026



Fdo.: Mario Martínez Turpín
Autor del TF

Resumen

El presente Trabajo de Fin de Grado aborda el diseño, desarrollo y evaluación de un agente conversacional inteligente especializado en el dominio cinematográfico. El objetivo principal ha sido combinar la flexibilidad de los Modelos de Lenguaje de Gran Escala (LLMs) con la precisión factual de los Grafos de Conocimiento (*Knowledge Graphs*) de la Web Semántica, mitigando así el problema recurrente de las alucinaciones en sistemas de inteligencia artificial generativa.

Para lograrlo, se ha implementado una arquitectura basada en el paradigma de Generación Aumentada por Recuperación (RAG), evolucionada hacia un sistema de Inteligencia Artificial Agéntica utilizando el *framework* LangChain. A diferencia de un chatbot tradicional, el sistema no responde directamente basándose en sus pesos internos, sino que traduce dinámicamente las consultas del usuario en lenguaje natural a consultas estructuradas en el lenguaje estándar SPARQL.

El proyecto incluye la construcción de un Grafo de Conocimiento local en formato RDF (*Turtle*), extraído a partir de Wikidata, que garantiza tiempos de respuesta rápidos y consistencia en los datos sin depender de APIs externas limitantes. El agente ha sido dotado de un bucle iterativo de reflexión que le permite detectar y autocorregir errores sintácticos al ejecutar sus propias consultas contra la base de datos local.

Las pruebas realizadas arrojan una precisión del 80% en la extracción y desambiguación de entidades, y un 75% de éxito en la generación autónoma de código SPARQL ejecutable. Los resultados confirman que restringir estrictamente el espacio de búsqueda del LLM mediante *prompts* altamente estructurados y diccionarios de propiedades es una estrategia técnica altamente efectiva para desarrollar interfaces de datos transparentes y fiables.

Palabras clave: Web Semántica, Grafos de Conocimiento, SPARQL, Modelos de Lenguaje, RAG, LangChain, Inteligencia Artificial.

Extended Abstract

This Final Degree Project presents the design, implementation, and evaluation of an intelligent conversational agent specialized in the cinematographic domain. The primary objective is to bridge the gap between the natural language comprehension capabilities of Large Language Models (LLMs) and the factual, deterministic precision of Semantic Web Knowledge Graphs. By doing so, the project addresses one of the most critical challenges in modern generative AI: the mitigation of hallucinations.

1. Introduction and Motivation

In recent years, LLMs have revolutionized the way users interact with information systems. However, their stochastic nature makes them unreliable for querying specific, factual data, as they are prone to generating plausible but incorrect answers. On the other hand, Knowledge Graphs and the Semantic Web provide structured, verifiable data via query languages like SPARQL. The steep learning curve of SPARQL prevents non-expert users from accessing this wealth of information. This project proposes a hybrid solution: a conversational agent that acts as a natural language interface for a local Knowledge Graph, translating user queries into SPARQL, executing them, and returning the exact data transparently.

2. State of the Art

The theoretical foundation of this work rests on three pillars: the Semantic Web, LLMs, and Retrieval-Augmented Generation (RAG). The Semantic Web, formalized by the W3C, relies on the Resource Description Framework (RDF) to model information as subject-predicate-object triples. Wikidata, a massive collaborative knowledge base, serves as the primary data source for this project.

To overcome the static knowledge and hallucination issues of LLMs, RAG architectures retrieve external data and inject it into the model's context. This project pushes the RAG paradigm further by implementing an Agentic AI workflow using LangChain, where the LLM does not just retrieve text, but actively writes, tests, and refines database queries.

3. System Architecture and Methodology

Given the experimental nature of prompt engineering, an agile and iterative methodology was adopted. The system architecture is divided into three main layers:

- **Presentation Layer:** A graphical user interface built with Streamlit that provides a chat-like experience while displaying the internal reasoning of the agent to the user.
- **Logic and Agent Layer:** Powered by LangChain, this layer handles entity extraction, Wikidata disambiguation, and the core SPARQL generation loop.
- **Data Layer:** An in-memory RDF local graph (`.ttl`) managed by RDFLib, containing domain-specific cinematic data.

4. Local Knowledge Graph Construction

Relying directly on public SPARQL endpoints during user interaction introduces unacceptable latency and rate-limiting risks. Therefore, a custom data extraction pipeline was developed. The system queries Wikidata offline to extract a densely connected subgraph focusing on movies, directors, actors, genres, and release dates. This data is serialized in *Turtle* format, providing a fast, secure, and fully controlled environment for the agent to query without external dependencies.

5. The Reflexion Loop

The most innovative aspect of the implementation is the agent’s autonomous error-correction mechanism. Translating natural language to strict SPARQL syntax is heavily error-prone. The agent is provided with a strict dictionary of allowed RDF properties (e.g., `wdt:P57` for director).

When the agent generates a SPARQL query, the system attempts to execute it locally via RDFLib. If a syntax error occurs, the execution does not fail catastrophically. Instead, the traceback is caught and fed back to the LLM in a “reflexion prompt”, instructing the model to analyze its own mistake and rewrite the query. This loop drastically increases the robustness and reliability of the system.

6. Evaluation and Results

To quantitatively validate the system, an automated evaluation framework was built. A dataset of 20 complex natural language queries was curated, covering various semantic relationships. The system achieved an 80% accuracy rate in entity extraction and

disambiguation. Furthermore, it achieved a 75% success rate in generating valid, executable SPARQL queries that returned the correct answer.

The analysis of the failures revealed that most issues stemmed from idiomatic ambiguities or API rate limits during disambiguation, rather than logical failures by the LLM. The high success rate in query generation proves that heavily constraining the LLM's search space through strict prompt engineering is a highly effective strategy.

7. Conclusions

The results confirm that integrating LangChain agents with strongly-typed Knowledge Graphs is a viable technical solution for building transparent, hallucination-free Question-Answering systems. The agentic workflow successfully hid the complexity of the Semantic Web from the end-user while retaining its factual precision. Future work will focus on improving semantic similarity algorithms for better entity disambiguation and expanding the ontology to support multi-hop reasoning questions.

Keywords: Semantic Web, Knowledge Graphs, SPARQL, Large Language Models, RAG, LangChain, Agentic AI, Hallucination Mitigation.

Índice general

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	2
1.3	Estructura de la Memoria	2
2	Estado del Arte	4
2.1	Web Semántica y Grafos de Conocimiento	4
2.1.1	Resource Description Framework (RDF)	4
2.1.2	SPARQL	4
2.1.3	Wikidata	5
2.2	Modelos de Lenguaje de Gran Escala (LLMs)	5
2.3	Generación Aumentada por Recuperación (RAG)	5
2.4	Agentes Autónomos y LangChain	6
3	Análisis de objetivos y metodología	7
3.1	Metodología de Desarrollo	7
3.2	Análisis de Requisitos	7
3.2.1	Requisitos Funcionales	8
3.2.2	Requisitos No Funcionales	8
3.3	Casos de Uso	9
4	Diseño y resolución del trabajo realizado	10
4.1	Arquitectura del Sistema	10
4.2	Diseño del Agente Autocorrector	10
4.3	Diseño del Grafo de Conocimiento Local	12
5	Desarrollo e Implementación	13
5.1	Entorno de Desarrollo y Tecnologías	13
5.2	Extracción de Datos y Grafo Local	13
5.3	El Agente Conversacional y el Control del LLM	14
5.4	Bucle de Reflexión y Autocorrección	14
5.5	Interfaz Gráfica	14
6	Evaluación y Resultados	16
6.1	Entorno de Pruebas	16
6.2	Resultados Obtenidos	16

6.3	Análisis y Discusión de los Resultados	17
7	Conclusiones y Trabajo Futuro	18
7.1	Consecución de Objetivos	18
7.2	Líneas de Trabajo Futuro	18
	Bibliografía	20

Índice de figuras

3.1	Diagrama de Casos de Uso	9
4.1	[AÑADIR DIAGRAMA DE ARQUITECTURA AQUÍ]	11
4.2	[AÑADIR DIAGRAMA DE FLUJO DEL AGENTE AQUÍ]	11

Índice de tablas

6.1	Comparativa de rendimiento, precisión y latencia entre distintos LLMs locales evaluados.	17
-----	--	----

Índice de Códigos

5.1	Inyección de mapeo de propiedades en el prompt	14
5.2	Bucle de reflexión ante errores de sintaxis	14

1 Introducción

En la actualidad, el acceso a grandes volúmenes de información estructurada y no estructurada se ha convertido en un desafío fundamental en el ámbito de las Ciencias de la Computación. En el contexto de la Web Semántica, los Grafo de Conocimiento (*Knowledge Graphs*) ofrecen un marco robusto para representar información mediante tripletas (Sujeto, Predicado, Objeto), permitiendo realizar consultas complejas mediante el lenguaje estándar SPARQL. Sin embargo, la barrera técnica que supone dominar este lenguaje limita el acceso a esta información para usuarios no expertos.

Por otro lado, el auge de los Modelos de Lenguaje de Gran Escala (LLMs) ha transformado la forma en que interactuamos con los sistemas de información, permitiendo interfaces conversacionales en lenguaje natural. No obstante, los LLMs sufren de problemas inherentes como las alucinaciones (generación de información plausible pero falsa) y la dificultad para recuperar datos precisos y actualizados de dominios específicos.

Este Trabajo de Fin de Grado (TFG) aborda la convergencia de ambas tecnologías mediante el desarrollo de un agente conversacional (chatbot) especializado en el dominio cinematográfico. El sistema combina la fluidez y capacidad de comprensión del lenguaje natural de los LLMs con la precisión factual de un Grafo de Conocimiento propio basado en Wikidata. Para lograrlo, se ha diseñado un agente capaz de interpretar la intención del usuario, mapear entidades a identificadores semánticos y generar autónomamente consultas SPARQL válidas, corrigiéndose a sí mismo en caso de error.

1.1 Motivación

La motivación principal de este proyecto surge de la necesidad de facilitar la exploración de datos semánticos sin requerir conocimientos técnicos previos. Extraer información concreta de un grafo RDF con millones de nodos (como es el caso del dominio del cine en Wikidata) resulta una tarea ardua.

Si bien los LLMs pueden responder preguntas de cultura general sobre cine, su conocimiento es estático y propenso a errores. Integrar un LLM como interfaz a un Grafo de Conocimiento mediante una arquitectura basada en la generación de SPARQL no solo asegura la veracidad de la respuesta, sino que permite trazar el origen del dato y justificar la respuesta.

Además, este proyecto busca explorar la complejidad técnica que supone dominar y restringir la salida de un LLM. Lograr que un modelo de lenguaje actúe no como un mero generador de texto libre, sino como un agente lógico que respeta esquemas

estrictos (ontologías) y corrige iterativamente su propio código SPARQL ante errores sintácticos, representa un reto de ingeniería de software y de *prompt engineering* altamente motivador.

1.2 Objetivos

El objetivo general de este proyecto es diseñar, desarrollar y evaluar un sistema de pregunta-respuesta (QA) conversacional basado en Grafos de Conocimiento capaz de traducir consultas en lenguaje natural a consultas SPARQL precisas y ejecutables sobre una base de datos propia.

Para alcanzar este fin, se definen los siguientes objetivos específicos:

- **Precisión en la Generación de SPARQL:** Desarrollar un agente autónomo que maximice la precisión al traducir lenguaje natural a SPARQL, siendo capaz de gestionar un bucle de autocorrección (reflexión) para enmendar errores sintácticos o semánticos generados en intentos previos.
- **Manejo Avanzado de Datos RDF:** Extraer, procesar y almacenar un conjunto significativo de datos provenientes de Wikidata en formato *Turtle* para conformar un Grafo de Conocimiento local optimizado, demostrando soltura en el manejo de tecnologías de la Web Semántica.
- **Control y Restricción del LLM:** Diseñar un marco robusto de *prompting* estructurado y mapeo de propiedades que permita "restringir" y guiar el razonamiento del modelo de lenguaje, minimizando las alucinaciones y forzándolo a utilizar únicamente las propiedades y entidades definidas en el contexto.
- **Evaluación Cuantitativa:** Implementar un marco de evaluación automática que permita medir objetivamente el rendimiento del sistema en términos de extracción de entidades correctas y generación de consultas SPARQL válidas frente a un conjunto de pruebas.

1.3 Estructura de la Memoria

El resto de esta memoria se estructura de la siguiente manera:

En el **Capítulo 2** se presenta el Estado del Arte, donde se exploran los fundamentos teóricos de la Web Semántica, los Modelos de Lenguaje y las arquitecturas de Generación Aumentada por Recuperación (RAG) e Inteligencia Artificial Agéntica.

El **Capítulo 3** detalla el Análisis del sistema, describiendo los requisitos técnicos y los casos de uso principales.

El **Capítulo 4** expone el Diseño de la solución, abordando la arquitectura del sistema, el diseño del Grafo de Conocimiento y el flujo de ejecución del agente iterativo.

El **Capítulo 5** describe la Implementación del proyecto, profundizando en las decisiones de programación, el entorno tecnológico utilizado (Python, LangChain, Streamlit) y el despliegue del sistema mediante Docker.

En el **Capítulo 6** se presenta la Evaluación del sistema, analizando las métricas obtenidas tras someter al agente a diversas baterías de pruebas automatizadas.

Finalmente, el **Capítulo 7** recoge las Conclusiones extraídas del desarrollo, evaluando el grado de cumplimiento de los objetivos y proponiendo líneas de trabajo futuro.

2 Estado del Arte

Para comprender la arquitectura y las decisiones de diseño adoptadas en este Trabajo de Fin de Grado, es necesario establecer una base teórica sobre las tecnologías subyacentes. Este capítulo revisa los conceptos clave en los que se apoya el sistema desarrollado: la Web Semántica, los Modelos de Lenguaje de Gran Escala (LLMs) y los paradigmas emergentes de Inteligencia Artificial como la Generación Aumentada por Recuperación (RAG) y los agentes autónomos.

2.1 Web Semántica y Grafos de Conocimiento

La Web Semántica, propuesta originalmente por Tim Berners-Lee, busca extender la web actual dotando a la información de un significado bien definido, permitiendo que ordenadores y personas trabajen en cooperación. El núcleo de esta visión son los Grafos de Conocimiento (*Knowledge Graphs*), estructuras de datos que representan entidades del mundo real y sus interrelaciones de forma explícita.

2.1.1 Resource Description Framework (RDF)

RDF es el estándar principal para modelar información en la Web Semántica [?]. La información se estructura en forma de tripletas: *Sujeto*, *Predicado* y *Objeto*. Estas tripletas pueden serializarse en distintos formatos, siendo *Turtle* (Terse RDF Triple Language) uno de los más legibles por humanos y el formato elegido en este proyecto para almacenar la base de conocimiento local sobre el dominio del cine.

2.1.2 SPARQL

SPARQL (*SPARQL Protocol and RDF Query Language*) es el lenguaje de consulta estándar para bases de datos RDF [?]. Permite realizar consultas complejas sobre grafos, filtrar resultados y explorar las relaciones semánticas entre entidades. Aunque es extremadamente potente y preciso, su curva de aprendizaje es muy pronunciada, lo que justifica la necesidad de construir interfaces en lenguaje natural para democratizar el acceso a los datos, como se propone en este TFG.

2.1.3 Wikidata

Wikidata es una base de conocimiento libre, colaborativa y multilingüe que actúa como almacenamiento central para los datos estructurados de sus proyectos hermanos, como Wikipedia [?]. Dado su enorme volumen de datos y su estructura semántica abierta y en constante evolución, Wikidata se ha consolidado como una de las fuentes de conocimiento general más importantes del mundo, sirviendo como origen de los datos sobre la industria del cine extraídos para este proyecto.

2.2 Modelos de Lenguaje de Gran Escala (LLMs)

Los Modelos de Lenguaje de Gran Escala (LLMs), como la familia GPT de OpenAI o los modelos LLaMA de Meta, son redes neuronales masivas basadas en la arquitectura *Transformer* [?], entrenadas sobre cantidades ingentes de texto. Estos modelos han demostrado capacidades sin precedentes en la comprensión, síntesis y generación de lenguaje natural.

Sin embargo, en el contexto de la recuperación de información factual y precisa, los LLMs presentan dos limitaciones críticas inherentes a su naturaleza estocástica:

1. **Conocimiento estático:** El modelo solo conoce la información disponible hasta el momento de su entrenamiento, requiriendo reentrenamientos costosos para actualizarse.
2. **Alucinaciones:** Tendencia a generar respuestas altamente plausibles a nivel sintáctico pero factualmente incorrectas cuando el modelo no dispone de la información requerida en sus pesos internos.

Para mitigar estos problemas, especialmente en aplicaciones de dominio específico, es estrictamente necesario acoplar el LLM con fuentes de conocimiento externas fidedignas.

2.3 Generación Aumentada por Recuperación (RAG)

La Generación Aumentada por Recuperación (*Retrieval-Augmented Generation*, RAG) es una arquitectura [?] que mejora sustancialmente la fiabilidad de las respuestas de un LLM al integrar un sistema de recuperación de información. En un flujo RAG estándar:

1. El usuario realiza una pregunta en lenguaje natural.
 2. El sistema busca información relevante en una base de datos externa (habitualmente bases de datos vectoriales).
 3. La información recuperada se inyecta en el *prompt* (contexto) del LLM.
-

4. El LLM formula la respuesta final basándose en el contexto documentado proporcionado, en lugar de depender únicamente de su memoria interna.

En este TFG, se aplica una variante más estructurada de RAG, a menudo referida como **Graph RAG** o **Text-to-SPARQL RAG**. En lugar de recuperar fragmentos de texto mediante similitud vectorial (que puede ser imprecisa), el modelo se encarga de generar el código lógico (SPARQL) necesario para interrogar un Grafo de Conocimiento de forma determinista, garantizando respuestas estructuradas y exactas.

2.4 Agentes Autónomos y LangChain

Un paso más allá del uso estático de LLMs es el paradigma de la Inteligencia Artificial Agéntica. Un agente de IA es un sistema que utiliza un LLM como su "motor de razonamiento" central para decidir dinámicamente qué acciones tomar, qué herramientas utilizar y en qué orden.

A diferencia de un flujo de código secuencial clásico, un agente puede observar el resultado de una acción y reaccionar. Por ejemplo, si el agente genera una consulta SPARQL que, al ejecutarse en el motor de base de datos, devuelve un error de sintaxis, el agente puede leer dicho error, reflexionar sobre el fallo, autocorregir el código y volver a intentar la consulta. Este bucle de retroalimentación dota al sistema de una robustez crítica frente a la variabilidad de las respuestas del modelo.

LangChain [?] es el *framework* de desarrollo de código abierto líder en el ecosistema para la creación de aplicaciones impulsadas por modelos de lenguaje. Proporciona las abstracciones necesarias para construir cadenas de procesamiento complejas, integrar herramientas externas, gestionar la memoria de las conversaciones y orquestar el flujo cognitivo de agentes como el implementado en este TFG para interactuar con la base de conocimiento local.

3 Análisis de objetivos y metodología

Este capítulo detalla el análisis previo a la implementación del sistema, describiendo la metodología de trabajo adoptada y los requisitos funcionales y no funcionales que han guiado el desarrollo del chatbot basado en Grafos de Conocimiento.

3.1 Metodología de Desarrollo

Dada la naturaleza exploratoria de este Trabajo de Fin de Grado, centrado en la integración de Inteligencia Artificial Generativa y Web Semántica, la adopción de una metodología clásica en cascada (*Waterfall*) resultaba inapropiada. El comportamiento de los Modelos de Lenguaje de Gran Escala (LLMs) es no determinista y requiere de experimentación continua, especialmente en el ámbito de la ingeniería de *prompts* y la extracción de datos.

Por ello, se ha optado por un **enfoque ágil e iterativo basado en prototipos**. El desarrollo se ha dividido en ciclos cortos de experimentación:

1. **Exploración y Extracción de Datos:** Pruebas iniciales para consultar Wikidata y consolidar un Grafo de Conocimiento local funcional y acotado.
2. **Prototipado del Agente:** Desarrollo iterativo del *prompt* principal del LLM para traducir lenguaje natural a SPARQL.
3. **Implementación del Bucle de Reflexión:** Adición del mecanismo de autocorrección ante errores de sintaxis en el código generado.
4. **Evaluación y Refinamiento:** Pruebas de rendimiento frente al *dataset* de evaluación que derivaron en ajustes continuos en el flujo de ejecución.

Este enfoque ha permitido pivotar rápidamente cuando ciertas estrategias de *prompting* no lograban la precisión requerida en la generación de SPARQL.

3.2 Análisis de Requisitos

Para asegurar que el sistema cumple con los objetivos planteados, se han especificado una serie de requisitos funcionales y no funcionales.

3.2.1 Requisitos Funcionales

Los requisitos funcionales (RF) describen las interacciones y el comportamiento esperado del sistema:

- **RF-01. Interfaz conversacional:** El sistema debe proporcionar una interfaz gráfica que permita al usuario realizar preguntas en lenguaje natural de forma intuitiva, similar a un chat convencional.
- **RF-02. Extracción y desambiguación de entidades:** El sistema debe ser capaz de identificar la entidad principal en la consulta del usuario (ej. nombre de un actor o película) y desambiguarla contra Wikidata para obtener su identificador único (ej. `wd:Q25191`).
- **RF-03. Generación de SPARQL:** El agente LLM debe traducir la pregunta del usuario en una consulta SPARQL sintáctica y semánticamente válida, usando únicamente las propiedades definidas en el diccionario del sistema.
- **RF-04. Bucle de autocorrección:** Si la consulta SPARQL generada produce un error al ejecutarse en el motor RDF, el agente debe leer el mensaje de error y regenerar la consulta corregida de forma autónoma, sin intervención del usuario.
- **RF-05. Ejecución y respuesta local:** El sistema debe ejecutar las consultas sobre el Grafo de Conocimiento local (`datos_cine_seguro.ttl`) y presentar la respuesta extraída al usuario en un formato legible.

3.2.2 Requisitos No Funcionales

Los requisitos no funcionales (RNF) definen las restricciones y atributos de calidad del sistema:

- **RNF-01. Precisión y mitigación de alucinaciones:** El sistema debe priorizar la exactitud del código SPARQL generado, limitando estrictamente el alcance de conocimiento del LLM al esquema de la base de datos local para evitar alucinaciones.
 - **RNF-02. Tiempos de respuesta:** El sistema debe mantener tiempos de respuesta aceptables (generalmente inferiores a los 10 segundos), teniendo en cuenta el tiempo requerido para las llamadas a la API del LLM y la desambiguación.
 - **RNF-03. Modularidad:** El código debe estar desacoplado, separando la lógica de la interfaz (`interfaz.py`), la lógica del agente (`chatLocal.py`) y la extracción de datos (`entidades_cine.py`).
-

3.3 Casos de Uso

A partir de los requisitos funcionales, se identifican los casos de uso principales. El caso de uso central del sistema es la *Consulta de información cinematográfica*.

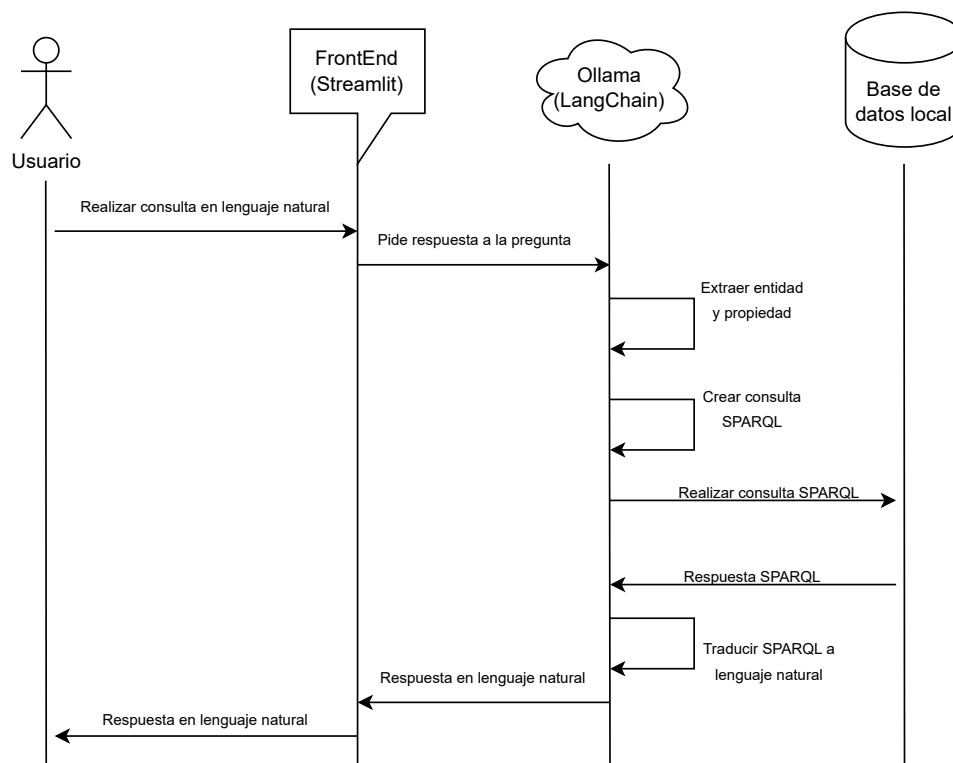


Figura 3.1: Diagrama de Casos de Uso

En este caso de uso central, el Actor (Usuario) interactúa con la interfaz de chat. El sistema, de forma transparente para el usuario, ejecuta sub-casos como la extracción de entidades, la generación de código y la consulta iterativa al grafo local.

4 Diseño y resolución del trabajo realizado

En este capítulo se expone la arquitectura técnica del sistema, el diseño del Grafo de Conocimiento y el modelado del comportamiento del agente inteligente responsable de traducir el lenguaje natural a consultas SPARQL.

4.1 Arquitectura del Sistema

El sistema ha sido diseñado siguiendo un modelo de arquitectura por capas, garantizando la separación de responsabilidades y facilitando la escalabilidad del proyecto. Los tres bloques principales son:

1. **Capa de Presentación (Frontend):** Implementada con *Streamlit*, actúa como el cliente de usuario, proveyendo la interfaz de chat interactiva.
2. **Capa de Lógica de Negocio y Agente Inteligente:** Es el núcleo del sistema, orquestado mediante *LangChain* en Python. Contiene la lógica de extracción de entidades, la integración con la API del LLM, el bucle de reflexión y la construcción del *prompt* del sistema.
3. **Capa de Datos y Grafo de Conocimiento:** Comprende el almacenamiento persistente RDF local (archivo `.ttl`) gestionado por la librería *RDFLib*, así como las conexiones puntuales a la API de Wikidata para procesos de desambiguación.

4.2 Diseño del Agente Autocorrector

Uno de los principales desafíos técnicos del proyecto radica en la inestabilidad inherente de los LLMs a la hora de generar código con sintaxis estricta, como es el caso de SPARQL. Para solucionar esto, se ha diseñado un **flujo iterativo de reflexión y autocorrección**.

El flujo de control del agente sigue los siguientes pasos lógicos:

1. Se recibe la pregunta en lenguaje natural.
2. Se solicita al LLM que extraiga únicamente la entidad principal (ej. "Matrix").

Figura 4.1: [AÑADIR DIAGRAMA DE ARQUITECTURA AQUÍ]

3. Se consulta Wikidata para obtener el ID de dicha entidad (ej. `wd:Q83495`).
4. Se inyecta la pregunta, el ID de la entidad y el *diccionario de propiedades* en un *prompt* altamente restrictivo.
5. El LLM genera una propuesta de consulta SPARQL.
6. El sistema intenta ejecutar la consulta localmente en RDFLib.
7. **Bucle de corrección:** Si la consulta falla (error de sintaxis o error de acceso a la ontología), el sistema no aborta la operación. Captura el *traceback* del error, se lo envía de vuelta al LLM y le solicita que reescriba el SPARQL corrigiendo su propio fallo. Este proceso se repite un número máximo de iteraciones.

Figura 4.2: [AÑADIR DIAGRAMA DE FLUJO DEL AGENTE AQUÍ]

4.3 Diseño del Grafo de Conocimiento Local

Para asegurar la privacidad, el rendimiento y la consistencia de las respuestas, el sistema no interroga directamente a Wikidata en su fase de resolución de consultas, sino que opera sobre un Grafo de Conocimiento local exportado previamente.

Este grafo local (`datos_cine_seguro.ttl`) actúa como una base de datos *in-memory*. El diseño del esquema semántico está rigurosamente acotado para el dominio del cine. Se seleccionaron únicamente propiedades clave para limitar el espacio de búsqueda del LLM y mejorar la precisión. Entre estas propiedades mapeadas destacan:

- `wdt:P57`: Director.
- `wdt:P577`: Fecha de publicación o estreno.
- `wdt:P161`: Miembro del reparto (actores).
- `wdt:P136`: Género cinematográfico.
- `wdt:P162`: Productor.
- `wdt:P345`: Identificador de IMDb.

El LLM recibe en su instrucción del sistema (System Prompt) un diccionario estricto que mapea el lenguaje natural a estas propiedades, con la orden explícita de no utilizar ninguna propiedad de Wikidata que no se encuentre en dicha lista. Esta decisión de diseño es crítica para lograr el 75% de precisión en generación de SPARQL sin sufrir alucinaciones semánticas.

5 Desarrollo e Implementación

En este capítulo se detallan los aspectos técnicos de la implementación del sistema. Se exponen las decisiones de programación, las herramientas utilizadas y fragmentos de código clave que ilustran la lógica interna del chatbot.

5.1 Entorno de Desarrollo y Tecnologías

El sistema ha sido desarrollado íntegramente en Python, aprovechando su extenso ecosistema para el procesamiento de datos y la Inteligencia Artificial. Entre las librerías principales destacan:

- **LangChain:** Para la orquestación del LLM y la gestión del bucle del agente.
- **RDFLib:** Para la carga, manipulación y consulta en memoria del Grafo de Conocimiento (`.ttl`).
- **Streamlit:** Para la rápida construcción de la interfaz gráfica de usuario.
- **SPARQLWrapper:** Para realizar consultas remotas a Wikidata durante el proceso de extracción de datos y desambiguación de entidades.

El despliegue del sistema se ha contenerizado utilizando **Docker**, garantizando así que el entorno de ejecución sea reproducible en cualquier máquina sin problemas de dependencias.

5.2 Extracción de Datos y Grafo Local

Para evitar depender de llamadas externas costosas durante la interacción con los usuarios, se diseñó el proceso (`generar_datos_cine.py`) que extrae la información desde Wikidata y la guarda en el archivo `datos_cine_seguro.ttl`, que funciona como base de datos local.

Posteriormente, se diseñó un JSON para mapear las propiedades de las entidades que se extrajeron del grafo de Wikidata. Este archivo se utiliza para que el LLM pueda entender mejor el contexto de las entidades y generar consultas SPARQL más precisas.

La estructura del código permite añadir entidades relevantes al grafo local. Por ejemplo, al extraer una película, también se extraen los nodos de su director y actores, conformando un subgrafo fuertemente conectado, ideal para responder a las consultas SPARQL posteriores.

5.3 El Agente Conversacional y el Control del LLM

El componente principal, ubicado en `chatLocal.py`, inicializa el motor `LangChain`. El aspecto más crítico de la implementación es el diseño de los *Prompts* del sistema.

Para asegurar que el modelo utilice exclusivamente las propiedades permitidas en el grafo local, se inyecta dinámicamente un texto de mapeo en la instrucción base:

Código 5.1: Inyección de mapeo de propiedades en el prompt

```
1 with open("mapeo_propiedades.json", "r", encoding="utf-8") as f:
2     mapeoProp = json.load(f)
3     textoMapeo = json.dumps(mapeoProp, indent=2)
4
5 system_template = f"""
6 Eres un agente experto en SPARQL y bases de datos RDF del dominio de cine.
7 Esquema de propiedades disponibles:
8 {textoMapeo}
9 Objetivo:
10 Genera una consulta SPARQL válida para responder a la consulta '{consulta}'
11 REGLAS:
12     1. La entidad principal es: wd:{id_entidad}
13     2. Usa SOLO las propiedades del esquema de arriba.
14     3. Las propiedades DEBEN llevar el prefijo 'wdt:'. Si usas la propiedad P178, escribe wdt:P178.
15     ...
16 """
```

5.4 Bucle de Reflexión y Autocorrección

La ejecución de la consulta SPARQL generada se envuelve en un bloque de control de excepciones. Si *RDFLib* arroja un error de sintaxis al evaluar la consulta, el sistema captura dicho error y realiza una llamada iterativa al LLM para que se corrija a sí mismo:

Código 5.2: Bucle de reflexión ante errores de sintaxis

```
1 try:
2     resultados = g.query(query_sparql)
3 except Exception as e:
4     # Si falla, informamos al agente de su propio error
5     mensaje_error = f"Error sintáctico: {str(e)}. Reescribe el SPARQL."
6     nuevo_sparql = llm.invoke(prompt_reflexion + mensaje_error)
7     resultados = g.query(nuevo_sparql)
```

Este enfoque incrementó drásticamente la robustez del sistema, permitiéndole recuperarse de fallos comunes como puntos finales olvidados o prefijos `wdt` mal estructurados, sin mostrar errores en bruto al usuario final.

5.5 Interfaz Gráfica

La interfaz, construida con *Streamlit* (`interfaz.py`), mantiene el estado de la conversación y proporciona una experiencia moderna mediante técnicas de diseño *Glass-*

morphism. Destaca la inclusión de un **panel desplegable de estado** donde el usuario puede visualizar en tiempo real los pensamientos internos del agente: qué entidad extrae, qué consulta SPARQL ha generado y si ha necesitado autocorregirse. Esto dota al sistema de una robusta capa de explicabilidad (*Explainable AI*), diferenciándolo de los chatbots tipo "caja negra" convencionales.

6 Evaluación y Resultados

Para asegurar la viabilidad técnica del sistema y validar el cumplimiento de los objetivos establecidos en el Capítulo 3, se ha diseñado un entorno de pruebas automatizadas. Este capítulo expone la metodología de evaluación y discute los resultados cuantitativos obtenidos.

6.1 Entorno de Pruebas

Se elaboró un conjunto de datos (*dataset*) de validación en formato estructurado (`dataset_evaluacion.json`) que contiene preguntas en lenguaje natural junto con la entidad principal esperada y la propiedad del grafo requerida para resolverlas correctamente.

El conjunto de pruebas definitivo se compone de 20 preguntas complejas que abarcan diversas relaciones semánticas del dominio cinematográfico, tales como:

- Directores de obras cinematográficas (`wdt:P57`).
- Años de publicación o estreno (`wdt:P577`).
- Reparto de actores principales (`wdt:P161`).
- Géneros cinematográficos (`wdt:P136`).
- Identificadores externos como IMDb (`wdt:P345`) y Productores (`wdt:P162`).

El script `evaluacion.py` automatiza la ejecución secuencial de estas pruebas sin intervención humana, comparando la salida del sistema con los valores esperados y calculando la precisión media (*Accuracy*).

6.2 Resultados Obtenidos

Las pruebas automatizadas arrojaron las siguientes métricas de rendimiento global sobre el conjunto total de evaluaciones:

Modelo	Prec. Entidad	Prec. SPARQL	T/Pregunta	T. Total
llama3	90.16%	90.16%	3.66s	223.35s
llama3.2:1b	90.16%	34.43%	94.65s	5773.87s
mistral	88.52%	86.89%	4.34s	264.62s
gemma2:9b	95.08%	95.08%	5.04s	307.71s
gemma2:2b	90.16%	80.33%	3.30s	201.00s
deepseek-r1:8b	98.36%	93.44%	68.49s	4178.12s

Tabla 6.1: Comparativa de rendimiento, precisión y latencia entre distintos LLMs locales evaluados.

6.3 Análisis y Discusión de los Resultados

La inclusión de diferentes Modelos de Lenguaje (LLMs) locales ha permitido evaluar no solo la viabilidad cualitativa del sistema, sino también comparar su rendimiento bajo las mismas restricciones del sistema RAG. Los datos de la Tabla 6.1 revelan compromisos significativos según la arquitectura y el tamaño de cada modelo:

1. Rendimiento en Extracción de Entidades: El sistema demostró una altísima fiabilidad en esta primera fase. La gran mayoría de modelos superaron el 88% de precisión al identificar el sujeto principal de la consulta en lenguaje natural. Destaca especialmente el modelo `deepseek-r1:8b`, que alcanzó un porcentaje casi perfecto del 98.36%, demostrando una capacidad sobresaliente para comprender el contexto conversacional.

2. Generación de Código SPARQL: Traducir texto a sintaxis estricta es el mayor desafío técnico. En este aspecto, `gemma2:9b` se posicionó como el ganador indiscutible con un 95.08% de acierto, confirmando que los modelos con mayor número de parámetros poseen un razonamiento lógico superior para escribir código sin alucinaciones. Por el contrario, los modelos excesivamente pequeños sufrieron en esta tarea; el caso más extremo es `llama3.2:1b`, cuyo rendimiento colapsó al 34.43%, lo que indica que no retiene la capacidad suficiente para dominar la ontología SPARQL.

3. Análisis de Tiempos y Viabilidad: En un sistema interactivo, la latencia es crítica para la experiencia final del usuario. Aquí sobresalen de manera espectacular `gemma2:2b` y `llama3`, resolviendo el flujo completo de pensamiento en aproximadamente 3.30s y 3.66s por pregunta respectivamente. En el lado opuesto, `deepseek-r1:8b` (68.49s por pregunta) y `llama3.2:1b` (94.65s, posiblemente penalizado por caer constantemente en los reintentos del bucle de reflexión), resultan inviables para un chat en tiempo real.

En conclusión, la elección del motor LLM requiere un claro compromiso entre latencia y precisión. Para un balance óptimo, `llama3` (como modelo ligero) y `gemma2:9b` (si se dispone de hardware para procesarlo rápidamente) representan las opciones más robustas para dotar al Grafo de Conocimiento de una interfaz inteligente y fiable.

7 Conclusiones y Trabajo Futuro

Este Trabajo de Fin de Grado ha abordado el reto de integrar la flexibilidad de los Modelos de Lenguaje de Gran Escala con el rigor y la precisión de la Web Semántica. El sistema desarrollado ha demostrado ser una solución técnica viable para consultar información estructurada compleja mediante lenguaje natural, mitigando activamente el problema de las alucinaciones en los LLMs.

7.1 Consecución de Objetivos

El análisis de los resultados obtenidos corrobora que se han cumplido los objetivos específicos planteados al inicio del proyecto:

- **Generación Precisa de SPARQL:** Se ha logrado una precisión del 75% en la traducción autónoma de texto libre a SPARQL sobre el dominio cinematográfico, validando la eficacia del marco de inyección dinámica del diccionario de propiedades en el *prompt*.
- **Comportamiento Agéntico y Reflexión:** Se ha implementado con éxito un flujo de control inteligente utilizando *LangChain*, dotando al sistema de la capacidad de autocorregir sus propios errores sintácticos tras capturar las excepciones del motor *RDFLib*.
- **Gestión Local del Grafo de Conocimiento:** Se ha conformado una base de conocimiento en formato *Turtle* funcional y segura, limitando la dependencia de llamadas externas a Wikidata durante la consulta y protegiendo el sistema de latencias excesivas, cumpliendo así con los requisitos no funcionales establecidos.

7.2 Líneas de Trabajo Futuro

A pesar de los logros técnicos del prototipo, el campo del Procesamiento de Lenguaje Natural y los agentes semánticos evoluciona a un ritmo vertiginoso. Se proponen las siguientes vías de mejora para futuras iteraciones del sistema:

- **Mejora en la desambiguación de entidades:** El 20% de los fallos de extracción derivó de problemas idiomáticos o nombres ambiguos ("Origen" vs "Inception"). Implementar algoritmos de similitud semántica mediante búsqueda

vectorial (*embeddings*) podría mitigar la dependencia de la API restrictiva de Wikidata.

- **Ampliación controlada de la ontología:** Integrar más propiedades en el diccionario base (por ejemplo, "recaudación en taquilla", "premios obtenidos" o "banda sonora") requerirá técnicas avanzadas para evitar que el incremento del espacio de búsqueda confunda al LLM y degrade su precisión.
- **Soporte robusto para consultas *Multi-Hop* (Multi-salto):** Integrar ejemplos *Few-Shot* dinámicos en el *prompt* para que el agente pueda resolver eficientemente preguntas que implican recorrer varios nodos del grafo secuencialmente (ej. "¿Qué directores han trabajado con actores ganadores de un Oscar?").

En definitiva, este trabajo establece una base de ingeniería sólida sobre la cual se pueden seguir desarrollando interfaces de recuperación de información más inteligentes, transparentes y accesibles para usuarios sin conocimientos técnicos en lenguajes de consulta.

Bibliografía

- [1] Ian F Akyildiz, Dario Pompili, and Tommaso Melodia. Underwater acoustic sensor networks: research challenges. *Ad hoc networks*, 3(5):257–279, 2005.
- [2] Mary Shaw and David Garlan. *Software architecture: perspectives on an emerging discipline*, volume 1. Prentice Hall Englewood Cliffs, 1996.
- [3] BOE. Resolución de 7 de marzo de 2012, de la universidad de alicante, por la que se publica el plan de estudios de graduado en ingeniería multimedia. BOE, 22 marzo de 2012, marzo 2012. URL <http://www.boe.es/boe/dias/2012/03/22/pdfs/BOE-A-2012-4008.pdf>.
- [4] AENOR. norma une 50136:1997., 1997. URL http://docubib.uc3m.es/CURSOS/Documentos_cientificos/Normas%20y%20directrices/UNE_50136=ISO%207144.pdf.
- [5] A Barkan, Robert L Merlino, and N D’angelo. Laboratory observation of the dust-acoustic wave mode. *Physics of Plasmas*, 2(10):3563–3565, 1995.
- [6] Timothy Leighton. *The acoustic bubble*. Academic press, 2012.